

Software Engineer — Step 1

Programming Fundamentals

Roadmap objective: Develop general programming and problem-solving foundations

Level: Beginner → Intermediate

Last reviewed: September 2026

How to use these notes

Read the concepts first, reproduce the examples yourself, complete the practical lab, and answer the interview questions without looking at the notes. Use the official documentation links as the final authority for changing APIs and platform behavior.

Learning outcomes

- Variables/types
- Control flow
- Functions
- Data structures
- Algorithms
- Complexity
- Debugging
- Modularity
- Testing basics

Core concepts

1. Programming fundamentals are language-independent skills: decompose a problem, represent data, choose an algorithm, implement, test and debug.
2. Learn one primary language deeply enough to express algorithms without fighting syntax.
3. Functions and modules reduce complexity by creating clear boundaries.
4. Big-O describes how resource usage grows as input size grows; it is a comparison tool, not a stopwatch.

Concept map

```
Programming Fundamentals
|
+--> Learn the concept
|
+--> Reproduce a small example
|
+--> Apply it in a feature
|
+--> Test failure/edge cases
|
+--> Document the decision
```

Detailed study guide

1. Variables/types

Study variables/types through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

2. Control flow

Study control flow through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

3. Functions

Study functions through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

4. Data structures

Study data structures through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

5. Algorithms

Study algorithms through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

6. Complexity

Study complexity through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

Worked example

Algorithm skeleton

```
function findMax(numbers) {  
  let max = numbers[0];  
  
  for (const value of numbers) {  
    if (value > max) max = value;  
  }  
  
  return max;  
}  
  
Time: O(n)  
Extra space: O(1)
```

Practice variation: change one input, introduce one failure, predict the result, then run it. Rewrite the example from memory afterward.

Comparison table

Concept	Meaning	Practical use
Syntax	How to write	Language rules
Algorithm	How to solve	Procedure
Data structure	How to store	Representation
Complexity	How cost grows	Scalability reasoning

Common mistakes

- Ignoring edge cases
- Optimizing before correctness
- Using unclear variable names
- Not testing intermediate logic
- Confusing Big-O with exact runtime

Best practices

- Keep responsibilities clear and small.
- Validate inputs at system boundaries.
- Prefer readable code over clever code.
- Test important success and failure paths.
- Use official documentation for current API/platform behavior.
- Keep secrets and credentials out of source control.
- Document important trade-offs so another developer can reproduce your reasoning.

Extended practical lab

Complete these tasks in order. The goal is to turn passive knowledge into evidence that you can actually use the topic.

Lab 1

- Solve ten tiny problems without an IDE autocomplete crutch.
- Write down what you expected, what happened, and why.

Lab 2

- Rewrite one solution using a different data structure.
- Write down what you expected, what happened, and why.

Lab 3

- Estimate complexity before coding.
- Write down what you expected, what happened, and why.

Lab 4

- Write tests for boundary cases.
- Write down what you expected, what happened, and why.

Lab 5

- Refactor duplicated logic.
- Write down what you expected, what happened, and why.

Lab 6

- Explain the solution in plain language.
- Write down what you expected, what happened, and why.

Mini-project

Implement 10 small programming problems: max/min, frequency count, string reversal, search, sorting, input validation, file parsing, recursion and basic API-style data transformation.

Acceptance criteria

- A new developer can understand the project from its README.
- The main happy path works from start to finish.
- At least three edge/failure cases are tested.
- The project has meaningful Git commits.
- The project includes a short explanation of design choices.
- If deployment applies, production behavior is verified rather than assumed.

Interview preparation

Practice answering these aloud. A strong answer should define the concept, give a concrete example, mention one trade-off, and explain when you would use it.

What is an algorithm?

- Answer structure: definition → example → trade-off/limitation → practical use.

What is a data structure?

- Answer structure: definition → example → trade-off/limitation → practical use.

What is Big-O?

- Answer structure: definition → example → trade-off/limitation → practical use.

Why use functions?

- Answer structure: definition → example → trade-off/limitation → practical use.

How do you test edge cases?

- Answer structure: definition → example → trade-off/limitation → practical use.

How do you debug systematically?

- Answer structure: definition → example → trade-off/limitation → practical use.

Debugging checklist

- Reproduce the problem consistently.
- Reduce it to the smallest failing example.
- Inspect inputs at each boundary.
- Check the first incorrect value, not only the final error.
- Read the complete error message and stack trace.
- Change one thing at a time.
- Retest the original failure after the fix.
- Add a regression test when the bug is important.

Glossary

Term	Your own definition	Example/use
Variable	Write this in your own words.	Give one project example.
Type	Write this in your own words.	Give one project example.
Function	Write this in your own words.	Give one project example.
Algorithm	Write this in your own words.	Give one project example.
Data structure	Write this in your own words.	Give one project example.
Complexity	Write this in your own words.	Give one project example.
Iteration	Write this in your own words.	Give one project example.
Recursion	Write this in your own words.	Give one project example.

Term	Your own definition	Example/use
Test	Write this in your own words.	Give one project example.
Debugging	Write this in your own words.	Give one project example.

Self-test

- Explain Variables/types without using its textbook definition.
- Show a small example of Control flow.
- What is the most important boundary in this topic?
- What is one failure mode and how would you diagnose it?
- What trade-off should a developer know before using this technique?
- How would you test the normal path and an edge case?
- How does this topic connect to the next roadmap step?
- What would you change if the system had 100× more users/data?
- What part of this topic would you explain differently to a beginner?
- What can you now build that you could not build before studying this step?

Final checklist

- I can explain and demonstrate Variables/types.
- I can explain and demonstrate Control flow.
- I can explain and demonstrate Functions.
- I can explain and demonstrate Data structures.
- I can explain and demonstrate Algorithms.
- I can explain and demonstrate Complexity.
- I can explain and demonstrate Debugging.
- I can explain and demonstrate Modularity.
- I can explain and demonstrate Testing basics.
- I completed the practical lab.
- I built or extended the mini-project.
- I tested at least three failure/edge cases.
- I can explain one trade-off.
- I can answer the interview questions without notes.
- I know where to find the authoritative documentation.

Recommended documentation

- <https://developer.mozilla.org/>
- <https://docs.python.org/3/>

Recommended videos

- Modern JavaScript Course (Existing DB resource 98)
- Java Full Course - freeCodeCamp (Existing DB resource 112)

Source note: resource references are based on the existing CareerCompass database plus verified web research. For changing APIs, versions, deployment settings and security guidance, consult the linked official documentation before production use.